
Speculative Decoding: What’s Measured, What’s Missed, and What’s Next

Lily Zhang

Madison Kanna

Abstract

An interactive, plain-language resource that teaches speculative decoding as the field evaluates it, then examines the evaluation itself. Learners trace three generations of draft models (EAGLE-3, DFlash, DSpark), learn to read *acceptance length* correctly, and see why rejection-sampling verification makes the base method provably lossless. The standard harness never grades outputs, the theorem covers only exact verification, and the measured behavior gap between owner-trained and vendor-assembled acceleration reaches 5.6 points on long-tail domains. A one-GPU, one-afternoon lab lets learners reproduce acceptance numbers, sweep the confidence threshold that converts lossless into lossy, and grade the outputs themselves.

1 Concept Summary

Autoregressive LLM decoding is *memory-bandwidth bound*, not compute bound: generating one token streams the entire weight matrix from HBM while compute sits idle, so a forward pass over K tokens costs almost the same as a pass over one. Speculative decoding exploits this: a small draft model proposes K tokens, and the target model verifies all K in a single pass, accepting a correct prefix via rejection sampling so the output distribution matches the target exactly. Because correctness is guaranteed by construction, evaluation collapsed onto a single dimension, efficiency, reported as *acceptance length* and tokens/s. Draft training converged onto the same number: the LK loss is the negative log of per-token acceptance probability.

The interactive walkthrough carries one worked example across all four decoding stages and walks the three generations, each removing the previous bottleneck: **EAGLE-3** reuses target hidden states for feature-level drafting (~ 3.9 tokens/pass); **DFlash** drafts a whole block in one diffusion pass (~ 4.4); **DSpark** adds a sequential head and confidence-scheduled trimming (~ 5.1 ; Qwen3-4B code average, DSpark Table 1). The DSpark stage also shows where lossless ends: a low-confidence draft token is trimmed before verification and corrected by the target. The resource then asks what this measurement misses. In DeepSpec’s evaluation harness the nine standard datasets supply only *prompts*; no grader or accuracy metric exists in the code, so the lossless claim rests entirely on the theorem. That theorem covers exact verification only; quantization, KV routing, disaggregation, and relaxed acceptance sit outside it, and the gap is measurable (owner-trained acceleration lost 0.3 points on a behavior benchmark where a vendor-assembled stack lost 5.6). Acceptance is also domain-conditional: verification is densest in low-entropy math and code, exactly where acceleration is least likely to misbehave. The resource ends on the open question: no published objective trains a draft to preserve the target’s *behavior* on the long tail.

2 Prerequisites and Technical Level

Level: introductory-to-intermediate; accessible to any engineer or student who knows that an LLM generates text one token at a time. **Prerequisites:** basic familiarity with transformer inference (a forward pass produces a next-token distribution); no RL, training, or GPU-kernel background re-

quired. The rejection-sampling losslessness argument is presented intuitively, with the accept/repair rule optional for readers who want the math.

3 Learning Objectives and Outcomes

After engaging with this resource, a learner can:

- explain *why* speculative decoding is possible (verifying K tokens $\approx$ the cost of one), and read *acceptance length* correctly: accepted proposals per verification step, not saved passes;
- trace the progression from EAGLE-3 to DFlash to DSpark, name the bottleneck each stage removes, and explain why training objective and evaluation metric converged onto the same number;
- **demonstrate hands-on** that acceptance rate is not accuracy: sweep the confidence threshold in the official evaluation code, grade the outputs, and watch the two numbers move in opposite directions;
- articulate the domain mismatch, verification concentrates where acceptance is naturally highest (math, code) while usage concentrates elsewhere, and pose the open question: what would a draft-training objective that preserves long-tail behavior look like?

4 Grounding in NeurIPS Research (2022–2026)

Speculative decoding is an active NeurIPS research line:

- **SpecTr** (NeurIPS 2023): generalizes draft selection to k candidates via optimal transport, with further speedups over standard speculative decoding.
- **Sequoia** (NeurIPS 2024): a dynamic-programming algorithm for the optimal speculation *tree*, up to $9.5\times$ speedups.
- **SpecExec** (NeurIPS 2024): builds a cache tree validated in a single target pass, up to 20 tokens per iteration on offloaded consumer hardware.

The walkthrough positions **EAGLE-3** (2025), **DFlash** (2026), and **DSpark** (DeepSeek, 2026) as the applied frontier these methods enable, alongside frontier work cited throughout: the foundational speculative-decoding papers (Leviathan et al., 2023; Chen et al., 2023), FP8 pre-training in DeepSeek-V3 (2024), MXFP4 QAT in gpt-oss (2025), Judge Decoding (2025), and Kimi K3 shipping the speculator as part of post-training (2026).

5 Teaching Materials Summary

All materials are original and created for this track: (1) an **interactive self-contained HTML article** following the What’s Measured / What’s Missed / What’s Next arc, with a walkthrough with adjustable draft length; (2) a **one-afternoon lab** (single GPU): reproduce the reported EAGLE-3 acceptance length (2.4–2.8) on Qwen3-8B, compare the three released DeepSpec checkpoints on one prompt set, sweep `-confidence-threshold` while grading outputs for correctness, and a pick-a-domain exercise outside math and code; (3) a **short video walkthrough** recorded from the site. All figures are original; numbers cite the DSpark paper and the LosslessBench evaluation.